

Lazy Loading vs Eager Loading in Hibernate

In Hibernate, **lazy loading** and **eager loading** control how related entities are fetched from the database. The `get()`, `load()`, and `getReference()` methods play crucial roles in these processes, especially when dealing with proxy objects, transient objects, and the timing of data retrieval.

1. `get()` Method

- **Immediate Fetch** (Eager Loading): The `get()` method fetches the object from the database immediately, returning a fully initialized object.
- **Usage:** It is typically used when you need the actual object right away.
- **Return Type:** Returns null if no record is found.

Example:

```
Session session = sessionFactory.openSession();
Student student = session.get(Student.class, 1); // Eagerly loads the object
System.out.println("ID: " + student.getId());
System.out.println("Name: " + student.getName());
System.out.println("City: " + student.getCity());
```

- **Behavior:** The `get()` method sends an immediate SQL SELECT query to the database to retrieve the record. The object is fully initialized when fetched.
- **Use Case:** Use `get()` when you need immediate access to the object, like when displaying student details.

2. `load()` Method

- **Lazy Loading:** The `load()` method doesn't hit the database immediately. Instead, it returns a proxy object, and the actual database query is executed when you access a property of the entity.

- **Usage:** It is used when you might or might not need to access the object fields.
- **Return Type:** Throws an exception if the object is not found.

Example:

```
Session session = sessionFactory.openSession();
Student student = session.get(Student.class, 1); // Eagerly loads the object
System.out.println("ID: " + student.getId());
System.out.println("Name: " + student.getName());
System.out.println("City: " + student.getCity());
```

- **Behavior:** load() returns a proxy object (a placeholder object). The actual query to the database is triggered only when a method like getName() or getCity() is called.
- **Proxy Object:** If you try to print the proxy object directly, it won't show the actual values until a method is accessed. When accessing the object's properties, Hibernate initializes the object by executing the query.
- **Use Case:** Use load() when you might not need the object immediately or in scenarios where performance optimization is needed by delaying the database call.

3. getReference() Method

- **Lazy Loading:** The getReference() method replaces the deprecated load() method in newer versions of Hibernate, maintaining the same lazy loading behavior by returning a proxy object.
- The actual query is executed only when the properties of the object are accessed.
- **Usage:** Typically used in entity relationships.
- **Return Type:** Throws an exception if the object is not found.

Example:

```
Session session = sessionFactory.openSession();
```

```
Student student = session.getReference(Student.class, 1); // Proxy object
System.out.println("ID: " + student.getId());           // Executes query
```

- **Behavior:** Works similarly to `load()` by returning a proxy object. It triggers the database query only when the object properties are accessed.

4. Proxy Object Behavior

A **proxy object** is a placeholder object returned by Hibernate when using lazy loading (`load()` or `getReference()`). The proxy object doesn't contain the actual data; instead, the database is queried when an attribute (like `getName()` or `getCity()`) is accessed.

- **Accessing Proxy Object:** If the entity's properties (e.g., `getName()`, `getCity()`) are accessed, Hibernate triggers a database call to fetch the full object data. If the entity is **not found**, Hibernate throws an exception.
- **Proxy Object Example:**

```
Student student = session.load(Student.class, 1); // Returns a proxy object
System.out.println("ID: " + student.getId());     // Triggers actual database call
```

5. Transient Object

A **transient object** in Hibernate is an object that has not been associated with any database session. It doesn't have a corresponding row in the database.

- **Behavior with Transient Objects:** If a transient object is used with methods like `get()` or `load()`, Hibernate will not find it in the session or database, which might lead to exceptions.

6. Key Differences Between `get()`, `load()`, and `getReference()`

- **`get()`:** Always hits the database immediately. Returns null if the object is not found.
- **`load()`:** Returns a proxy object. Throws an exception if the object is not found.

- **getReference():** Similar to load(), returns a proxy object and throws an exception if the object is not found.

7. Example: Handling Proxy Object and Lazy Loading

```
Session session = sessionFactory.openSession();
try {
    // Using get()
    Student student = session.get(Student.class, 1);
    if (student != null) {
        System.out.println("ID: " + student.getId());
        System.out.println("Name: " + student.getName());
        System.out.println("City: " + student.getCity());
    } else {
        System.out.println("No data found with the given ID");
    }

    // Using load()
    //Student proxyStudent = session.load(Student.class, 2); // Lazy loading
    Student proxyStudent = session.getReference(Student.class, 2);
    System.out.println("Accessing proxy object: " + proxyStudent.getName());
} catch (HibernateException e) {
    e.printStackTrace();
} finally {
    session.close();
}
```

Summary of Key Points:

- **get():** Immediate data retrieval (eager loading), returns null if not found.
- **load() and getReference():** Lazy loading, returns a proxy object, triggers data fetch only when properties are accessed, and throws exceptions if not found.

- **Proxy Object:** Used in lazy loading to delay the database query. Hibernate provides an actual instance only when properties are accessed.
- **Transient Objects:** Objects not yet associated with the session, potentially leading to exceptions if accessed in certain ways.
- **Lazy Initialization Exception:** Be careful when closing the session before accessing lazy-loaded properties.